

嵌入式系统复习资料

杨子颀

2026 年 6 月 16 日

目录

1 第 1 部分：嵌入式系统概述	3
1.1 核心知识点	3
1.2 STM32 微处理器/微控制器内核	3
1.3 常用基础 API 与功能	4
2 第 2 部分：GPIO 引脚配置与基本输入输出	4
2.1 GPIO 核心知识点	4
2.2 推挽输出与开漏输出区别	5
2.3 GPIO 常用 API 与功能	5
3 第 3 部分：中断、NVIC 与 EXTI	5
3.1 中断基本概念	5
3.2 中断优先级与分组	6
3.3 EXTI 中断线映射	6
3.4 中断响应步骤与 HAL 处理流程	6
3.5 代码模板：按键控制 LED（查询方式）	7
4 第 4 部分：定时器基础与输出比较	8
4.1 定时器核心知识点	8
4.2 时基单元构成及作用	8
4.3 基本定时器与通用定时器	8
4.4 输出比较与 PWM 基础	9
4.5 代码模板：定时器周期中断	9
5 第 5 部分：串行通信与 DMA	10
5.1 同步通信与异步通信	10
5.2 异步串口 UART/USART	10
5.3 I2C 通信	11

5.4	SPI 通信	11
5.5	DMA 的作用	11
6	第 6 部分：A/D 转换 ADC	12
6.1	核心知识点	12
6.2	ADC 工作原理与特点	13
6.3	ADC 转换计算	13
6.4	ADC 常用 API 与功能	14
7	第 7 部分：RTC 与看门狗	14
7.1	RTC 实时时钟	14
7.2	看门狗	14
8	第 8 部分：时钟树与滴答定时器 SysTick	15
8.1	时钟树知识点	15
8.2	PLL 锁相环	15
8.3	SysTick 滴答定时器	15
9	第 9 部分：ADC 采集并通过串口打印	16
9.1	实现流程	16
9.2	串口发送与接收	16
9.3	片上温度采集	16
9.4	代码模板：ADC 采集电压并串口打印	17
10	第 10 部分：按键控制指示灯	17
10.1	接口电路与工作模式	17
10.2	GPIO 配置要点	18
10.3	代码模板：按键控制 LED	18
11	第 11 部分：PWM 原理与控制函数	18
11.1	PWM 基本原理	18
11.2	PWM 参数计算例题	19
11.3	代码模板：设置 PWM 占空比	19
11.4	控制执行函数：判断超限报警	20
12	第 1-11 部分 API 总表	20
13	考试速记	22

1 第 1 部分：嵌入式系统概述

1.1 核心知识点

嵌入式系统是什么

嵌入式系统是以应用为中心、以计算机技术为基础，软硬件可裁剪，适用于特定功能、可靠性、成本、体积和功耗要求的专用计算机系统。它不是通用电脑，而是嵌入到具体设备中完成控制、采集、通信、显示、运算等任务。

嵌入式系统组成

- **硬件层：**微控制器/微处理器、存储器、GPIO、定时器、ADC、通信接口、电源、传感器、执行器等。
- **驱动层：**GPIO 驱动、串口驱动、定时器驱动、ADC 驱动、I2C/SPI 驱动等，负责直接操作硬件。
- **操作系统层：**裸机系统可以没有操作系统；复杂系统可使用 RTOS，实现任务调度、定时、信号量、消息队列等功能。
- **应用层：**完成具体产品功能，如温度采集、按键控制、数据显示、报警、通信上传等。

嵌入式系统特点

- **专用性强：**面向特定任务设计，如饮水机控制、心率检测、温度采集。
- **实时性要求：**需要在规定时间内响应外部事件，如按键、中断、报警、通信接收。
- **资源受限：**Flash、RAM、CPU 主频、IO 数量有限，程序要尽量精简。
- **可靠性高：**常用于长期运行场景，需要看门狗、异常处理、掉电保护等机制。
- **低功耗：**电池供电设备需要休眠、低功耗时钟、低功耗外设等设计。
- **软硬件结合紧密：**程序设计必须结合原理图、引脚、电平、时钟和外设特性。

1.2 STM32 微处理器/微控制器内核

STM32 与 Cortex-M 内核

STM32 是 ST 公司基于 ARM Cortex-M 内核设计的 32 位微控制器系列。常见 STM32F103 使用 Cortex-M3 内核，具有中断控制器 NVIC、SysTick 滴答定时器、低功耗模式、丰富外设接口等特点。复习时要知道：**Cortex-M 是内核，STM32 是在内核外围集成外设后的芯片系列。**

STM32 常见资源

- 内核资源：CPU、寄存器组、NVIC、SysTick、调试接口。
- 存储资源：Flash 用于存程序，SRAM 用于运行变量和堆栈。
- 通用外设：GPIO、EXTI、中断、定时器、ADC、DMA、RTC、看门狗。
- 通信外设：USART/UART、I2C、SPI、CAN、USB 等。

1.3 常用基础 API 与功能**易错点**

- 外设不工作时，先检查外设时钟是否使能。
- STM32 程序不是只写主函数，还要配置时钟、GPIO、外设、中断等硬件资源。
- HAL 库函数封装了寄存器操作，考试问“功能”时要能说清它背后完成的硬件配置。

2 第 2 部分：GPIO 引脚配置与基本输入输出**2.1 GPIO 核心知识点****GPIO 是什么**

GPIO（General Purpose Input Output，通用输入输出）是 MCU 与外部电路连接最常用的接口，可用于控制 LED、读取按键、驱动蜂鸣器、片选外设、作为复用外设引脚等。

GPIO 常见工作模式

- 输入模式：读取外部电平，可配合上拉、下拉或浮空输入。
- 推挽输出：既能输出高电平，也能输出低电平，驱动能力较强，LED 控制常用。
- 开漏输出：只能主动拉低，输出高电平需要外部或内部上拉，常用于 I2C 总线、线与逻辑、电平转换。
- 复用功能：引脚交给外设控制，如 USART、SPI、I2C、TIM PWM。
- 模拟模式：关闭数字输入输出缓冲，ADC 输入必须配置为模拟模式。

LED、按键、ADC 输入分别怎么配

- **LED**：一般配置为推挽输出，注意判断高电平点亮还是低电平点亮。
- **按键**：一般配置为输入模式；若按下接地，则常用上拉输入；若按下接电源，则常用下拉输入。
- **ADC 输入**：必须配置为模拟输入模式，不能配置为普通数字输入。

- **I2C 引脚**：常配置为复用开漏输出，并需要上拉电阻。
- **USART TX/PWM 输出**：常配置为复用推挽输出。

2.2 推挽输出与开漏输出区别

推挽与开漏比较

项目	推挽输出	开漏输出
输出高电平	可主动输出高电平	不能主动输出高电平，需要上拉电阻
输出低电平	可主动输出低电平	可主动拉低
驱动能力	较强，适合普通数字输出	高电平能力取决于上拉电阻
典型应用	LED、继电器控制、普通输出、USART TX	I2C、线与、多设备共享总线、电平转换

2.3 GPIO 常用 API 与功能

易错点

- GPIO 端口时钟没开，后面初始化和读写都可能无效。
- 按键输入要结合外部电路判断按下是高电平还是低电平。
- ADC 引脚不要配置成上拉输入，应配置为模拟模式。

3 第 3 部分：中断、NVIC 与 EXTI

3.1 中断基本概念

中断是什么

中断是 MCU 响应外部事件或内部外设事件的一种机制。当事件发生时，CPU 暂停当前程序，转去执行中断服务函数，处理完成后再返回原程序继续执行。常见中断来源包括按键外部中断、定时器溢出中断、串口接收中断、ADC 转换完成中断等。

NVIC 作用

NVIC (Nested Vectored Interrupt Controller, 嵌套向量中断控制器) 用于管理 Cortex-M 内核中断，包括中断使能、优先级设置、中断嵌套、挂起状态等。优先级数值越小，优先级越高。

3.2 中断优先级与分组

抢占优先级与响应优先级

- 抢占优先级：决定一个中断能否打断另一个正在执行的中断。
- 响应优先级/子优先级：当多个同抢占优先级中断同时等待时，决定先响应哪一个。
- 优先级判断：先比较抢占优先级，数值小者先执行；抢占优先级相同再比较子优先级，数值小者先执行。
- 优先级分组：把可用优先级位划分给抢占优先级和子优先级，不同分组会影响可设置的级数。

3.3 EXTI 中断线映射

外部中断线连接关系

外部 GPIO 中断通过 EXTI 线进入 NVIC。一般 Px0 对应 EXTI0，Px1 对应 EXTI1，以此类推到 EXTI15。同一条 EXTI 线同一时间只能选择一个端口来源，例如 PA0 和 PB0 都对应 EXTI0，不能同时作为两个独立 EXTI0 使用。STM32F1 中常通过 AFIO 配置 EXTI 端口映射。

常见 IRQ 通道

- EXTI0_IRQn：对应 EXTI0。
- EXTI1_IRQn：对应 EXTI1。
- EXTI2_IRQn：对应 EXTI2。
- EXTI3_IRQn：对应 EXTI3。
- EXTI4_IRQn：对应 EXTI4。
- EXTI9_5_IRQn：EXTI5 到 EXTI9 共用一个中断入口。
- EXTI15_10_IRQn：EXTI10 到 EXTI15 共用一个中断入口。

3.4 中断响应步骤与 HAL 处理流程

中断响应步骤

1. 外设或外部引脚产生中断请求，中断挂起标志置位。
2. NVIC 判断该中断是否使能，并比较优先级。
3. CPU 保存现场，把当前寄存器压栈。
4. CPU 根据中断向量表跳转到对应中断服务函数。

5. 在中断服务函数中调用 HAL 处理函数并清除中断标志。
6. 执行用户回调函数，完成具体任务。
7. 中断返回，CPU 恢复现场，继续执行原程序。

HAL 中断处理流程

HAL 库通常采用“中断入口函数 → HAL 统一处理函数 → 用户回调函数”的结构。例如外部中断常见流程为：

EXTI_IRQHandler() → HAL_GPIO_EXTI_IRQHandler() → HAL_GPIO_EXTI_Callback()

这种方式的优点是代码结构统一、可移植性较好、用户只需在回调函数中写业务逻辑。

3.5 代码模板：按键控制 LED（查询方式）

```
1 while (1)
2 {
3     if (HAL_GPIO_ReadPin(KEY_GPIO_Port, KEY_Pin) == GPIO_PIN_RESET)
4     {
5         HAL_Delay(20); // 消抖
6
7         if (HAL_GPIO_ReadPin(KEY_GPIO_Port, KEY_Pin) == GPIO_PIN_RESET)
8         {
9             HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
10
11             // 等待按键松开，防止一次按下触发多次
12             while (HAL_GPIO_ReadPin(KEY_GPIO_Port, KEY_Pin) ==
13                     GPIO_PIN_RESET);
14         }
15     }
```

易错点

- 中断优先级数字越小，优先级越高。
- 只配置 GPIO 中断模式还不够，还要使能 NVIC 对应 IRQ。
- 共用中断入口的 EXTI 线，要在中断处理函数中判断具体是哪一条线触发。

4 第4部分：定时器基础与输出比较

4.1 定时器核心知识点

定时器作用

定时器本质上是一个按时钟脉冲自动计数的硬件计数器。它可用于定时延时、周期中断、PWM 输出、输入捕获、输出比较、编码器接口、测频测速等功能。

三种计数模式

- 向上计数：CNT 从 0 计到 ARR，产生更新事件后回到 0。
- 向下计数：CNT 从 ARR 计到 0，产生更新事件后重新装载。
- 中央对齐计数：CNT 先向上计数再向下计数，常用于对称 PWM，能改善电机控制或功率控制中的波形特性。

4.2 时基单元构成及作用

时基单元

- PSC 预分频器：对输入定时器时钟分频，降低计数频率。
- CNT 计数器：保存当前计数值。
- ARR 自动重载寄存器：决定计数上限或周期。
- 更新事件：CNT 溢出或到达重载条件时产生，可触发中断、DMA 或更新寄存器。
- CCR 捕获/比较寄存器：用于输出比较、PWM 占空比或输入捕获值。

定时周期计算

若定时器输入时钟为 f_{TIM} ，预分频值为 PSC ，自动重载值为 ARR ，则定时器更新频率为：

$$f_{update} = \frac{f_{TIM}}{(PSC + 1)(ARR + 1)}$$

定时周期为：

$$T = \frac{(PSC + 1)(ARR + 1)}{f_{TIM}}$$

4.3 基本定时器与通用定时器

不同定时器功能比较

类型	主要功能	典型用途
基本定时器	只有基本计数、更新事件，部分芯片可触发 DAC	简单定时、周期中断、触发事件
通用定时器	基本计数、输入捕获、输出比较、PWM、编码器接口等	PWM、测频、测速、脉宽测量、周期任务
高级定时器	在通用定时器基础上增加互补输出、死区、刹车等	电机控制、逆变器、功率驱动

4.4 输出比较与 PWM 基础

输出比较功能

输出比较是把计数器 CNT 与比较寄存器 CCR 的值进行比较，当两者满足设定关系时，定时器改变输出电平或产生中断。PWM 就是输出比较功能的典型应用。

PWM 中 ARR 与 CCR 的作用

- **ARR**: 决定 PWM 周期。
- **PSC**: 决定计数速度。
- **CCR**: 决定有效电平持续时间，即占空比。
- **占空比**: $Duty = CCR / (ARR + 1) \times 100\%$ 。

4.5 代码模板：定时器周期中断

```

1 HAL_TIM_Base_Start_IT(&htim2);
2
3 void TIM2_IRQHandler(void)
4 {
5     HAL_TIM_IRQHandler(&htim2);
6 }
7
8 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
9 {
10     if (htim->Instance == TIM2)
11     {
12         HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
13     }

```

14

}

5 第 5 部分：串行通信与 DMA

5.1 同步通信与异步通信

同步与异步区别		
项目	同步通信	异步通信
时钟线	通常有独立时钟线	通常没有独立时钟线
同步方式	发送方提供或共享时钟	双方约定波特率，通过起始位和停止位同步
典型接口	SPI、I2C	UART/USART 异步模式
优点	速度较高，时序明确	接线少，使用简单，适合远距离低速通信

5.2 异步串口 UART/USART

波特率与字符帧格式	
• 波特率：单位时间内传输的符号数，常见为 9600、115200 等。 • 字符帧：通常由 1 位起始位、数据位、可选校验位、停止位组成。 • 常见格式：115200-8-N-1，表示 115200 波特率、8 位数据位、无校验、1 位停止位。 • 连接方式：A 板 TX 接 B 板 RX，A 板 RX 接 B 板 TX，双方需要共地。	
常见串口标志位	
• TXE：发送数据寄存器空，可以写入新数据。 • TC：发送完成，表示移位寄存器中的数据也发送结束。 • RXNE：接收数据寄存器非空，表示收到数据可读。 • IDLE：总线空闲标志，常配合 DMA 接收不定长数据。 • ORE：溢出错误，表示旧数据未及时读取又来了新数据。	

5.3 I2C 通信

I2C 线数与作用

I2C 通常使用两根线：SCL 为串行时钟线，SDA 为串行数据线。两根线一般为开漏结构，需要上拉电阻。I2C 支持一主多从，通过从机地址选择通信对象。

I2C 协议要点

- 起始信号：SCL 为高电平时，SDA 从高变低。
- 停止信号：SCL 为高电平时，SDA 从低变高。
- 数据有效性：SCL 高电平期间 SDA 必须保持稳定；SCL 低电平期间 SDA 才允许变化。
- 应答 ACK：接收方在第 9 个时钟周期拉低 SDA 表示应答。

5.4 SPI 通信

SPI 线数与作用

SPI 常用四根线：SCK 为时钟线，MOSI 为主机输出从机输入，MISO 为主机输入从机输出，NSS/CS 为片选信号。SPI 可全双工通信，速度通常高于 I2C。

SPI 四种时序模式

SPI 模式由 CPOL 和 CPHA 决定：

- **Mode 0**：CPOL=0，CPHA=0，空闲低电平，第一个边沿（上升沿）采样。
- **Mode 1**：CPOL=0，CPHA=1，空闲低电平，第二个边沿（下降沿）采样。
- **Mode 2**：CPOL=1，CPHA=0，空闲高电平，第一个边沿（下降沿）采样。
- **Mode 3**：CPOL=1，CPHA=1，空闲高电平，第二个边沿（上升沿）采样。

主从设备的 CPOL、CPHA 必须一致，否则会出现数据错位。

5.5 DMA 的作用

DMA 是什么

DMA（Direct Memory Access，直接存储器访问）可以在不占用 CPU 大量参与的情况下，在外设与内存、内存与内存之间搬运数据。它的核心作用是降低 CPU 负担，提高连续数据传输效率。ADC 连续采样、串口大量收发、SPI 屏幕刷新等场景常用 DMA。

易错点

- UART 通信时 TX 要接对方 RX，RX 要接对方 TX。
- I2C 的 SDA、SCL 是开漏结构，通常必须有上拉电阻。

- SPI 主从两端的 CPOL 和 CPHA 必须一致。
- DMA 只是搬运数据，外设本身仍需要正确初始化和启动。

6 第 6 部分：A/D 转换 ADC

6.1 核心知识点

ADC 是什么

ADC（Analog to Digital Converter，模数转换器）把连续变化的模拟电压转换成离散的数字量。STM32 内部 ADC 常用于采集电压、电流、温度、传感器输出等信号。

常考技术指标

- **分辨率**：ADC 能输出的数字位数，例如 12 位 ADC 输出范围为 0 ~ 4095。位数越高，理论上能区分的电压越细。
- **量化单位/最小分辨电压**：
$$\Delta V = \frac{V_{ref}}{2^N - 1}$$
其中 N 为 ADC 位数， V_{ref} 为参考电压。
- **转换精度**：实际转换值与真实模拟值的接近程度，受参考电压、噪声、非线性误差、采样时间等影响。
- **转换时间**：完成一次采样和转换所需时间，通常由采样周期和转换周期共同决定。
- **输入范围**：通常为 0 ~ V_{ref} ，输入电压不能超过芯片允许范围。
- **采样时间**：采样保持电容对输入信号采样的时间。信号源阻抗较大时，应适当加长采样时间。

A/D 转换基本步骤

1. 配置 ADC 时钟、GPIO 模拟输入模式、ADC 参数和通道。
2. 启动 ADC 转换。
3. 等待转换完成标志置位，或使用中断/DMA 等方式等待完成。
4. 读取 ADC 转换结果寄存器对应的数字值。
5. 按公式把数字量换算为实际电压或物理量。

6.2 ADC 工作原理与特点

工作原理

STM32 常用逐次逼近型 ADC。它会把输入模拟电压与内部 DAC 产生的比较电压逐位比较，逐步确定二进制结果。逐次逼近型 ADC 速度较快、功耗较低，适合 MCU 内部集成。

常见转换方式

- 单次转换：启动一次只采集一次，适合偶尔读取传感器。
- 连续转换：一次启动后持续采集，适合实时监测。
- 扫描模式：按顺序转换多个通道。
- 规则通道：普通采样序列，最常用。
- 注入通道：优先级更高，适合需要打断规则转换的采样任务。
- 查询方式：CPU 等待转换完成，简单但占用 CPU。
- 中断方式：转换完成后进入回调函数，CPU 占用较少。
- DMA 方式：转换结果自动搬运到内存，适合连续高速采样。

6.3 ADC 转换计算

数字量换算电压

若 ADC 分辨率为 N 位，参考电压为 V_{ref} ，读取值为 D ，则输入电压近似为：

$$V_{in} = \frac{D}{2^N - 1} \times V_{ref}$$

12 位 ADC 常写为：

$$V_{in} = \frac{D}{4095} \times V_{ref}$$

例如 $V_{ref} = 3.3 \text{ V}$ ， $D = 2048$ ，则：

$$V_{in} \approx \frac{2048}{4095} \times 3.3 \approx 1.65 \text{ V}$$

易错点

- 12 位 ADC 最大值是 4095，不是 4096。公式分母通常写 $2^N - 1$ 。
- ADC 引脚要配置为模拟输入模式，不要配置成普通输入或上拉输入。
- ADC 输入不能超过参考电压和芯片引脚允许电压，否则可能损坏芯片。
- 采样源阻抗过大时，采样时间太短会导致读数偏差。

7 第 7 部分：RTC 与看门狗

7.1 RTC 实时时钟

RTC 作用

RTC（Real-Time Clock，实时时钟）用于记录实际日期和时间，可实现年月日、时分秒、闹钟、时间戳、低功耗唤醒等功能。它常配合备份域和备用电池使用，使系统掉电后仍能保持时间。

RTC 工作时钟源

常见 RTC 时钟源有：

- **LSE**：外部低速晶振，典型频率 32.768 kHz，精度较高，RTC 最常用。
- **LSI**：内部低速 RC 振荡器，成本低，但精度较差。
- **HSE 分频**：由外部高速时钟分频得到，部分型号支持，但掉电保持和低功耗场景不如 LSE 常用。

7.2 看门狗

看门狗作用

看门狗用于防止程序跑飞或死机。程序正常运行时需要定期“喂狗”；如果程序异常导致超时未喂狗，看门狗会复位系统，提高系统可靠性。

IWDG 与 WWDG 区别

项目	IWDG 独立看门狗	WWDG 窗口看门狗
时钟源	LSI 内部低速时钟，独立于主时钟	APB 时钟分频，依赖系统时钟
特点	独立性强，适合系统级故障恢复	要求在规定时间窗口内喂狗，检测更严格
是否能关闭	一旦启动通常不能停止，直到复位	可通过配置控制
典型用途	防止程序死机、跑飞	检测过早或过晚喂狗等时序异常

8 第 8 部分：时钟树与滴答定时器 SysTick

8.1 时钟树知识点

系统时钟源

STM32 时钟树用于把不同振荡源经过选择、倍频、分频后提供给 CPU、总线和外设。常见系统时钟源包括：

- **HSI**：内部高速 RC 时钟，启动快、无需外部器件，但精度一般。
- **HSE**：外部高速晶振或外部时钟，精度较高。
- **PLL**：锁相环，可把 HSI 或 HSE 倍频，提高系统主频。

总线与外设时钟

- **SYSCLK**：系统时钟，作为 CPU 和总线时钟的来源。
- **HCLK**：AHB 总线时钟，通常给内核、存储器、DMA 等使用。
- **PCLK1**：APB1 外设时钟，常给低速外设使用，如 TIM2、USART2、I2C 等。
- **PCLK2**：APB2 外设时钟，常给高速外设使用，如 GPIO、ADC、USART1、TIM1 等。
- **定时器时钟**：当 APB 分频系数不为 1 时，部分 STM32 定时器时钟可能为 PCLK 的 2 倍。

8.2 PLL 锁相环

PLL 的作用

PLL 用于把输入时钟倍频，得到更高的系统时钟。例如 STM32F103 常见配置为 HSE = 8 MHz，PLL 倍频 9，得到 SYSCLK = 72 MHz。

$$f_{SYSCLK} = f_{PLL_IN} \times PLLMUL$$

8.3 SysTick 滴答定时器

SysTick 作用

SysTick 是 Cortex-M 内核自带的 24 位递减计数器，常用于产生 1 ms 系统节拍。HAL 库中的 HAL_Delay() 和 HAL_GetTick() 通常依赖 SysTick。

SysTick 常考点

- SysTick 属于内核外设，不属于普通通用定时器 TIMx。
- HAL 工程中通常每 1 ms 进入一次 SysTick_Handler()。

- 中断服务函数中调用 `HAL_IncTick()`，系统毫秒计数加 1。
- 使用 RTOS 时，SysTick 也可能作为操作系统节拍源。

9 第 9 部分：ADC 采集并通过串口打印

9.1 实现流程

ADC 采集串口打印流程

1. 配置 ADC 引脚为模拟输入，配置 ADC 通道和采样时间。
2. 配置 USART 的 TX/RX 引脚复用功能、波特率、数据位、停止位和校验位。
3. 调用 `HAL_ADC_Start()` 启动采样。
4. 调用 `HAL_ADC_PollForConversion()` 等待转换完成。
5. 调用 `HAL_ADC_GetValue()` 读取 ADC 值。
6. 用公式换算成电压或温度。
7. 用 `sprintf()` 格式化字符串，再用 `HAL_UART_Transmit()` 发送。

9.2 串口发送与接收

串口配置要点

- TX 为发送脚，RX 为接收脚。两块板通信时，一般 TX 接对方 RX，RX 接对方 TX，共地。
- 常用配置：115200 波特率、8 位数据位、1 位停止位、无校验，即 115200-8-N-1。
- 查询发送适合简单打印；中断或 DMA 发送适合减少 CPU 阻塞。
- 若用 `printf()` 重定向到串口，MDK 中常需要勾选 **Use MicroLIB**，并实现 `fputc()`。

9.3 片上温度采集

片上温度通道

片上温度传感器通常使用 `ADC_CHANNEL_TEMPSENSOR`。以部分 STM32F1 为例，内部温度换算常见公式为：

$$T = \frac{V_{25} - V_{sense}}{Avg_Slope} + 25$$

其中 V_{25} 为 25 摄氏度下传感器输出电压， Avg_Slope 为平均斜率，具体参数应以对应芯

片手册为准。对 STM32F103 常见典型值可写成：

$$T \approx \frac{1.43 - V_{sense}}{0.0043} + 25$$

9.4 代码模板：ADC 采集电压并串口打印

```
1 uint32_t adc_value;
2 float voltage;
3 char msg[64];
4
5 HAL_ADC_Start(&hadc1);
6 if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
7 {
8     adc_value = HAL_ADC_GetValue(&hadc1);
9     voltage = adc_value * 3.3f / 4095.0f;
10    sprintf(msg, "ADC=%lu, V=%.2fV\r\n", adc_value, voltage);
11    HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 100);
12 }
13 HAL_ADC_Stop(&hadc1);
```

10 第 10 部分：按键控制指示灯

10.1 接口电路与工作模式

LED 接口电路

LED 通常由 GPIO 口、限流电阻、发光二极管组成。常见连接方式有：

- 高电平点亮：GPIO 输出高电平，电流从 GPIO 流向 LED 和 GND。
- 低电平点亮：LED 接到电源端，GPIO 输出低电平时灌电流，LED 点亮。

答题时要先判断电路是高电平有效还是低电平有效。

按键接口电路

按键常用上拉或下拉输入：

- 上拉输入：未按下为高电平，按下接地为低电平。
- 下拉输入：未按下为低电平，按下接电源为高电平。
- 消抖：按键机械抖动会造成多次触发，可用延时消抖、状态机消抖或定时扫描消抖。

10.2 GPIO 配置要点

GPIO 模式选择

- LED: 通常配置为推挽输出, 输出能力强, 可输出高/低电平。
- 按键: 根据电路配置为上拉输入、下拉输入或浮空输入。
- ADC 输入: 应配置为模拟输入, 避免数字输入缓冲影响采样。
- 复用功能: 串口、PWM 等外设控制引脚时需要配置为复用推挽或复用开漏。

10.3 代码模板: 按键控制 LED

```

1  if (HAL_GPIO_ReadPin(KEY_GPIO_Port, KEY_Pin) == GPIO_PIN_RESET)
2  {
3      HAL_Delay(20);
4      if (HAL_GPIO_ReadPin(KEY_GPIO_Port, KEY_Pin) == GPIO_PIN_RESET)
5      {
6          HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
7          while (HAL_GPIO_ReadPin(KEY_GPIO_Port, KEY_Pin) == GPIO_PIN_RESET
8              )
9              {
10                 /* wait for release */
11             }
12     }
13 }
```

11 第 11 部分: PWM 原理与控制函数

11.1 PWM 基本原理

PWM 是什么

PWM (Pulse Width Modulation, 脉宽调制) 通过改变高电平持续时间占整个周期的比例来控制等效输出。常用于 LED 调光、电机调速、蜂鸣器、舵机控制、功率变换器等。

周期、频率、占空比

定时器 PWM 的常用计算公式:

$$f_{PWM} = \frac{f_{TIM}}{(PSC + 1)(ARR + 1)}$$

$$T_{PWM} = \frac{1}{f_{PWM}}$$

$$Duty = \frac{CCR}{ARR + 1} \times 100\%$$

其中 f_{TIM} 为定时器输入时钟， PSC 为预分频值， ARR 为自动重装载值， CCR 为比较值。

PWM 输出比较原理

定时器计数器 CNT 从 0 计数到 ARR。当 CNT 与 CCR 比较时，输出电平按 PWM 模式发生变化。CCR 越大，高电平持续时间越长，占空比越大。

11.2 PWM 参数计算例题

例：72 MHz 定时器时钟，产生 1 kHz PWM

假设 $f_{TIM} = 72 \text{ MHz}$ ，希望 $f_{PWM} = 1 \text{ kHz}$ 。可取 $PSC = 71$ ，则定时器计数频率为：

$$\frac{72 \text{ MHz}}{71 + 1} = 1 \text{ MHz}$$

要得到 1 kHz：

$$ARR + 1 = \frac{1 \text{ MHz}}{1 \text{ kHz}} = 1000$$

所以 $ARR = 999$ 。若占空比为 50%，则：

$$CCR = (ARR + 1) \times 50\% = 500$$

11.3 代码模板：设置 PWM 占空比

```

1 void PWM_SetDuty(TIM_HandleTypeDef *htim, uint32_t channel, float duty)
2 {
3     uint32_t arr;
4     uint32_t ccr;
5
6     if (duty < 0.0f) duty = 0.0f;
7     if (duty > 100.0f) duty = 100.0f;
8
9     arr = __HAL_TIM_GET_AUTORELOAD(htim);
10    ccr = (uint32_t)((arr + 1) * duty / 100.0f);
11    __HAL_TIM_SET_COMPARE(htim, channel, ccr);
12 }
13
14 HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
15 PWM_SetDuty(&htim3, TIM_CHANNEL_1, 50.0f);

```

11.4 控制执行函数：判断超限报警

常见综合题思路

综合题可能要求“采集 ADC 值，换算物理量，判断是否超限，控制 LED/蜂鸣器/PWM”。答题时可按以下顺序写：

1. 采集：HAL_ADC_Start()、HAL_ADC_PollForConversion()、HAL_ADC_GetValue()。
2. 换算：数字量换成电压或温度、电流等物理量。
3. 判断：与阈值比较。
4. 执行：超限则 LED 亮、蜂鸣器响、PWM 占空比改变；未超限则恢复正常。
5. 打印：用串口输出状态，方便调试。

```

1 void Control_Alarm(float value, float limit)
2 {
3     if (value >= limit)
4     {
5         HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
6         PWM_SetDuty(&htim3, TIM_CHANNEL_1, 80.0f);
7     }
8     else
9     {
10        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
11        PWM_SetDuty(&htim3, TIM_CHANNEL_1, 0.0f);
12    }
13 }

```

12 第 1-11 部分 API 总表

模块	常用 API	主要功能
概述	HAL_Init()	初始化 HAL 库与系统节拍等基础环境
概述	SystemClock_Config()	配置系统时钟树，常由 CubeMX 生成
GPIO	HAL_GPIO_Init()	初始化引脚模式、上下拉和速度
GPIO	HAL_GPIO_ReadPin()	读取按键或输入电平
GPIO	HAL_GPIO_WritePin()	控制 LED 或普通输出电平
中断	HAL_NVIC_SetPriority()	设置中断抢占优先级和子优先级
中断	HAL_NVIC_EnableIRQ()	使能 NVIC 中断通道

模块	常用 API	主要功能
中断	HAL_GPIO_EXTI_Callback()	外部中断回调, 编写用户处理逻辑
定时器	HAL_TIM_Base_Start_IT()	启动定时器周期中断
定时器	HAL_TIM_PeriodElapsedCallback()	定时器更新中断回调
UART	HAL_UART_Transmit()	串口阻塞发送数据
I2C	HAL_I2C_Mem_Read()	从 I2C 器件寄存器读取数据
SPI	HAL_SPI_TransmitReceive()	SPI 全双工收发数据
DMA	HAL_DMA_Start_IT()	启动 DMA 传输并开启中断
ADC	HAL_ADC_Start()	启动 ADC 转换
ADC	HAL_ADC_PollForConversion()	等待转换完成
ADC	HAL_ADC_GetValue()	读取 ADC 数字量
ADC	HAL_ADC_Start_IT()	中断方式启动 ADC
ADC	HAL_ADC_Start_DMA()	DMA 方式采样并搬运数据
RTC	HAL_RTC_SetTime()	设置时间
RTC	HAL_RTC_GetTime()	读取时间
RTC	HAL_RTC_SetAlarm_IT()	设置闹钟中断
IWDG	HAL_IWDG_Init()	初始化独立看门狗
IWDG	HAL_IWDG_Refresh()	喂狗, 防止复位
RCC	HAL_RCC_OscConfig()	配置振荡源和 PLL
RCC	HAL_RCC_ClockConfig()	配置系统时钟和总线分频
SysTick	HAL_GetTick()	获取毫秒节拍
SysTick	HAL_Delay()	毫秒延时
UART	HAL_UART_Receive_IT()	串口中断接收
UART	HAL_UART_RxCpltCallback()	串口接收完成回调
GPIO	HAL_GPIO_TogglePin()	翻转 LED 状态
PWM	HAL_TIM_PWM_Start()	启动 PWM 输出
PWM	__HAL_TIM_SET_COMPARE()	修改 CCR, 改变占空比
PWM	HAL_TIM_PWM_Stop()	停止 PWM 输出

13 考试速记

一句话记忆

- ADC: 模拟电压变数字量, 核心公式是 $V = D/(2^N - 1) \times V_{ref}$ 。
- RTC: 掉电也能计时, 优先记 LSE 32.768 kHz。
- 看门狗: 程序要定期喂狗, 超时复位; IWDG 用 LSI, WWDG 有窗口限制。
- 时钟树: HSI/HSE 进 PLL, 倍频后得到 SYSCLK, 再分到 AHB/APB。
- SysTick: 内核 24 位定时器, 常产生 1 ms 节拍。
- ADC 串口打印: 启动、查询、读取、换算、格式化、发送。
- 按键 LED: 按键读输入, LED 写输出, 先判断高低电平有效。
- PWM: 频率看 PSC 和 ARR, 占空比看 CCR。

简答题

题目: 利用通用定时器产生 PWM 信号驱动电机, 用以调节智能小车的速度。要求信号的输出频率为 10 kHz, 初始占空比为 30%。若定时器输入时钟为 72 MHz, 请根据题意设置预分频器 PSC、自动重装载寄存器 ARR 的值及比较寄存器 CCR 的值, 并写出计算过程。

解: 定时器 PWM 输出频率公式为:

$$f_{PWM} = \frac{f_{TIM}}{(PSC + 1)(ARR + 1)}$$

已知 $f_{TIM} = 72 \text{ MHz}$, $f_{PWM} = 10 \text{ kHz}$, 所以:

$$(PSC + 1)(ARR + 1) = \frac{72 \times 10^6}{10 \times 10^3} = 7200$$

为了计算方便, 可令 $PSC = 71$, 则定时器计数频率为:

$$f_{CNT} = \frac{72 \text{ MHz}}{PSC + 1} = \frac{72 \text{ MHz}}{72} = 1 \text{ MHz}$$

因此:

$$ARR + 1 = \frac{1 \text{ MHz}}{10 \text{ kHz}} = 100$$

所以 $ARR = 99$ 。初始占空比为 30%, 比较寄存器满足:

$$Duty = \frac{CCR}{ARR + 1} \times 100\%$$

因此:

$$CCR = (ARR + 1) \times 30\% = 100 \times 0.3 = 30$$

答案：可设置 $PSC = 71$, $ARR = 99$, $CCR = 30$ 。此时 PWM 频率为 10 kHz，初始占空比为 30%。

说明：本题答案不唯一，只要满足 $(PSC + 1)(ARR + 1) = 7200$ 即可。